

BarberTime — Sprint 1: Arquitetura e Backend REST

Disciplina: LDAMD — PUC Minas

Projeto: BarberTime — agendamento em barbearia

Sprint: 1 — Backend REST + persistência

1. Domínio e perfis

Perfil	Papel
Cliente	Solicita horário com barbeiro; remarca ou cancela seus agendamentos
Barbeiro (prestador)	Visualiza fila <code>pending</code> e confirma agendamentos

Fluxo principal: cliente cria solicitação `pending` → barbeiro confirma → `confirmed` .

2. Stack técnica

Camada	Tecnologia
API	Node.js 18+ / Express 4
Persistência	PostgreSQL
Autenticação	JWT + bcrypt
Datas / grade	Luxon
Testes de API	Postman (postman/BarberTime.postman_collection.json)

3. Modelo de dados (schema)

Arquivo: `database/schema.sql` (+ patches em `database/patches/`).

Tabela `users`

- `id` , `email` , `password_hash` , `full_name` , `role` (`client` | `barber` | `admin`)

Tabela `barbers`

- `id` , `full_name` , `active` , `user_id` → `users.id`

Tabela appointments

- `id`, `client_id`, `barber_id`, `starts_at`, `ends_at`
 - `status`: `pending`, `confirmed`, `cancelled`, `completed`
 - Índice único parcial: (`barber_id`, `starts_at`) para status `pending` e `confirmed`
-

4. Endpoints REST (mínimo 4+)

Base: `http://localhost:3000/api/v1`

Método	Rota	Descrição
POST	<code>/auth/register</code>	Cadastro cliente + JWT
POST	<code>/auth/login</code>	Login (cliente ou barbeiro)
GET	<code>/barbers</code>	Lista barbeiros ativos
GET	<code>/availability/available-slots</code>	Horários livres por barbeiro e data
POST	<code>/appointments</code>	Criar agendamento (pending)
GET	<code>/appointments</code>	Listar agendamentos do cliente
PATCH	<code>/appointments/:id/cancel</code>	Cancelar
GET	<code>/barber/appointments</code>	Fila do barbeiro
PATCH	<code>/barber/appointments/:id/confirm</code>	Confirmar → <code>confirmed</code>

5. Regras de negócio (grade)

Configuradas via `.env`:

- Expediente padrão: **9h–20h**, slots de **30 min**
 - **Domingo** fechado (`CLOSED_ON_SUNDAY`)
 - **Almoço** 12h–13h sem atendimento
 - `startsAt` deve ser início de slot válido (consultar `/availability/available-slots`)
-

6. Estrutura do código (Clean Architecture)

```
src/  
  config/          - env, pool PostgreSQL  
  controllers/     - HTTP  
  services/        - regras de negócio  
  repositories/    - SQL  
  domain/          - schedulePolicy (grade pura)  
  routes/          - rotas Express  
  middlewares/     - auth, roles, erros
```

7. Como executar (Sprint 1)

```
npm install  
cp .env.example .env # ajustar DATABASE_URL  
npm run db:migrate  
npm run dev
```

Barbeiros seed: `joao.barbeiro@barbertime.seed` /
`maria.barbeira@barbertime.seed` — senha `senha123` .

8. Entregas Sprint 1 (checklist)

Entrega	Artefato
Backend REST funcional	<code>src/</code> + <code>README.md</code>
Banco documentado	<code>database/schema.sql</code>
Coleção de testes	<code>postman/BarberTime.postman_collection.json</code>
Organização em camadas	estrutura <code>src/</code> acima

A partir da Sprint 2, a documentação de MOM/RabbitMQ está em `docs/sprint2/` .